

Multi-layer Motion Planning with Kinodynamic and Spatio-Temporal Constraints

Jeel Chatrola*

jchatrola@wpi.edu

Worcester Polytechnic Institute
Worcester, MA, USA

Kevin Leahy

k Leahy@wpi.edu

Worcester Polytechnic Institute
Worcester, MA, USA

Abhiroop Ajith*

aajith@wpi.edu

Worcester Polytechnic Institute
Worcester, MA, USA

Constantinos Chamzas

cchamzas@wpi.edu

Worcester Polytechnic Institute
Worcester, MA, USA

Abstract

We propose a novel, multi-layered planning approach for computing paths that satisfy both kinodynamic and spatiotemporal constraints. Our three-part framework first establishes potential sequences to meet spatial constraints, using them to calculate a geometric lead path. This path then guides an asymptotically optimal sampling-based kinodynamic planner, which minimizes an STL-robustness cost to jointly satisfy spatiotemporal and kinodynamic constraints. In our experiments, we test our method with a velocity-controlled Ackerman-car model and demonstrate significant efficiency gains compared to prior art. Additionally, our method is able to generate complex path maneuvers, such as crossovers, something that previous methods had not demonstrated.

Keywords

Motion Planning, Signal Temporal Logic, Robotics

1 Introduction

Motion planning is a core problem in robotics spanning applications from autonomous cars to long-horizon manipulation.

Sampling-based planners [22] have shown great promise in efficiently computing motion plans including scenarios where kinodynamic constraints must be considered. Recent kinodynamic planners have shown both enhanced efficiency [23] and asymptotically optimal (AO) convergence for a given cost [7] enabling the efficient computation of trajectories for non-holonomic robots such as acceleration-bounded vehicles.

However, as robotics becomes more ubiquitous and tasks grow in complexity, a single motion plan often fails to satisfy all task-specific requirements. Increasingly, robotic tasks require additional constraints to be met. For example, a delivery robot might need to visit several different regions in a time-sensitive manner as shown in Figure 1. To efficiently encode these complex mission objectives, expressive and precise logic-based tools have been used to describe the desired behavior of the system. Significant research has focused on integrating these logic-based methods with motion planning to tackle complex tasks. For instance, Linear Temporal Logic (LTL) [26] has been widely used to encode sequential and safety requirements

[24], while Planning Domain Definition Language (PDDL) is widely used for task and motion planning tasks in manipulation [8].

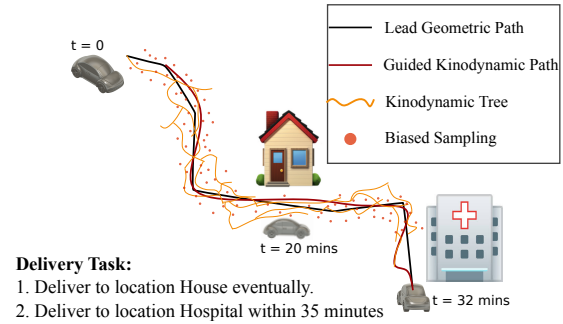


Figure 1: A delivery robot tasked with reaching different regions in a time sensitive-manner.

Signal Temporal Logic (STL) [20] has found widespread applicability, as it can quantitatively monitor the satisfaction of spatiotemporal requirements for system behavior across hybrid (discrete and continuous) domains. Typically, STL problems are formulated as Mixed Integer Linear Programs (MILP) [2, 27], which don't naturally handle kinodynamic planning problems. Thus, researchers [1, 11, 31] have started investigating encoding STL robustness as part of a cost function within asymptotically optimal planners, such as RRT* [10]. However, this only applies to simple kinodynamic models with easy to compute steering functions. Parallel work [9] drives the search with a time-partitioned STL automaton instead of a single robustness cost, but their method still requires a steering function—limiting applicability to complex dynamics.

To jointly address the challenges of spatiotemporal and kinodynamic constraints, we propose a novel multi-layer framework based on Stable Sparse Tree (SST)[18] that maintains probabilistic completeness and AO guarantees without requiring steering functions. Additionally, we introduce an efficient encoding scheme for the STL formulas that enables robots to handle complex spatio-temporal constraints using geometric leads. Our approach employs a decomposition method by first generating a sequence of potential spatial regions that need to be visited. Biased sampling is used to guide exploration through these regions, providing improved efficiency, while STL robustness is used as the cost of the SST planner.

* Equal contribution.

Code: <https://github.com/elpis-lab/LG-SST-STL>.

2 Related Work

Kinodynamic motion planning is challenging due to the complexities of differential constraints and high-dimensional state and control spaces. Finding an optimal control sequence to connect to points in the state-space is known as the 2-point-boundary problem (2BVP), which is impractical to compute multiple times [14]. Furthermore, the high dimensionality of the state and control spaces poses significant computational challenges, making it hard to efficiently explore and optimize solutions [15].

Over the years, several sampling-based planners [22] tailored towards kinodynamic-based systems have been proposed that do not require solving 2BVP problem. Examples include the original kinodynamic-RRT [15], and later layer-based improvements such as KPIECE [32] and SYCLOPS [25]. Additionally, Δ kinodynamic planners were developed that can optimize a given cost function such as edge-bundling planners [28] and the SST planner. However, these planners can only be used to solve a single motion plan, (e.g. reaching a single goal region) which does not suffice for more complex multi-goal missions such as in our setting.

To address this, researchers have leveraged practical tools and extensive literature from formal methods to augment motion planners to synthesize controllers for temporal logic specifications [12]. Methods most similar to our approach, which address spatio-temporal constraints, typically utilize STL formulae to specify a given task. These methods modify the cost function of the base planner to maximize the robustness of the STL formula, [11, 19, 31]. However, they rely on geometric Δ planners, which restrict their applicability to systems with simple dynamics. Furthermore, as STL formulae become more complex, encoding them into a single cost function becomes increasingly computationally expensive. In contrast, our work leverages kinodynamic Δ planners that do not require steering functions. Additionally, we propose a simple yet complete method to reduce the complexity of the optimized robustness function, enhancing computational efficiency without sacrificing completeness.

3 Preliminaries

Let $\mathbb{R}, \mathbb{R}_{\geq 0}, \mathbb{N}$ denote the set of real, non-negative real, and natural numbers, respectively. We denote by $\mathbb{R}^n$ an n -dimensional Euclidean space and by $\mathbb{R}^{n \times m}$ a space of real matrices with n rows and m columns. We use time intervals in the form $[a, b], a \leq b$. Further, we denote $t + [a, b]$ by $[t + a, t + b]$.

Let (M, d) be a compact metric space with $M \subset \mathbb{R}^n, n \geq 1$, and $\mathcal{S} = \{s : \mathbb{R}_{\geq 0} \rightarrow M\}$ the set of all infinite-time signals in M . The components of a signal $s \in \mathcal{S}$ are denoted by $s_i, i \in \{1, \dots, n\}$. The set of all linear functions over $\mathbb{R}^n$ is denoted by $\mathcal{F} = \{\pi : \mathbb{R}^n \rightarrow \mathbb{R}\}$.

The syntax of STL is defined as follows [20]:

$$\phi ::= \top \mid p_{\pi(x) \sim \mu} \mid \neg \phi \mid \phi_1 \wedge \phi_2 \mid \phi_1 \mathcal{U}_{[a,b]} \phi_2, \quad (1)$$

where $\top$ is the Boolean true constant; $p_{\pi(x) \sim \mu}$ is a predicate over $\mathbb{R}^n$ parameterized by $\pi \in \mathcal{F}, \mu \in \mathbb{R}$ and an order relation $\sim \in \{\geq, >, \leq, <\}$ of the form $p_{\pi(x) \sim \mu} = \pi(x) \sim \mu$; $\neg$ and $\wedge$ are the Boolean operators for negation and conjunction, respectively; and $\mathcal{U}_{[a,b]}$ is the bounded temporal operator *until*. The Boolean semantics of

STL is defined over signals in $\mathcal{S}$ recursively as follows [20]:

$$\begin{aligned} (s, t) &\models \top && \Leftrightarrow \top \\ (s, t) &\models p_{\pi(x) \geq \mu} && \Leftrightarrow \pi(s(0)) \geq \mu \\ (s, t) &\models p_{\pi(x) \leq \mu} && \Leftrightarrow \pi(s(0)) \leq \mu \\ (s, t) &\models \neg \phi && \Leftrightarrow \neg((s, t) \models \phi) \\ (s, t) &\models (\phi_1 \wedge \phi_2) && \Leftrightarrow (s \models \phi_1) \wedge ((s, t) \models \phi_2) \\ (s, t) &\models (\phi_1 \mathcal{U}_{[a,b]} \phi_2) && \Leftrightarrow \exists t' \in [a, b]. s.t. s(t') \models \phi_2 \wedge \\ &&& \forall t'' \in [0, t') s(t'') \models \phi_1, \end{aligned} \quad (2)$$

A signal $s \in \mathcal{S}$ is said to satisfy an STL formula ϕ if and only if $s(0) \models \phi$. The Boolean value false $\perp \equiv \neg \top$ and additional operations (i.e., disjunction, implication, and equivalence) are defined in the usual way. Also, the temporal operators eventually and globally are defined as $F_{[a,b]} \phi \equiv \top \mathcal{U}_{[a,b]} \phi$ and $G_{[a,b]} \phi \equiv \neg F_{[a,b]} \neg \phi$, respectively. In addition to Boolean semantics, STL admits quantitative semantics [5, 6], which are formalized by the notion of the robustness degree. The robustness degree of a signal $s \in \mathcal{S}$ with respect to an STL formula ϕ is a functional $\rho(s, \phi)$ recursively defined as:

$$\begin{aligned} \rho(s, p_{\pi(x) \geq \mu}, t) &= (\pi(s(0)) - \mu) \\ \rho(s, p_{\pi(x) \leq \mu}, t) &= (\mu - \pi(s(0))) \\ \rho(s, \neg \phi, t) &= -\rho(s, \phi, t) \\ \rho(s, \phi_1 \wedge \phi_2, t) &= \min \{ \rho(s, \phi_1, t), \rho(s, \phi_2, t) \} \\ \rho(s, \phi_1 \vee \phi_2, t) &= \max \{ \rho(s, \phi_1, t), \rho(s, \phi_2, t) \} \\ \rho(s, \phi_1 \mathcal{U}_{[a,b]} \phi_2, t) &= \max_{t_u \in [a,b]} \{ \min \{ \rho(s, \phi_2, t), \\ &\quad \min_{t' \in [0, t_u]} \{ \rho(s, \phi_1, t') \} \} \} \\ \rho(s, F_{[a,b]} \phi, t) &= \max_{t_u \in [a,b]} \{ \rho(s, \phi, t) \} \\ \rho(s, G_{[a,b]} \phi, t) &= \min_{t_u \in [a,b]} \{ \rho(s, \phi, t) \}. \end{aligned} \quad (3)$$

The robustness degree is sound, i.e., $s \models \phi \iff \rho(s, \phi, t) \geq 0$.

4 Problem Statement

4.1 Kinodynamic Constraints

Let $S = (f, X, U, x_{init})$ be a dynamical system, where $X \subseteq \mathbb{R}^n$ and $U \subseteq \mathbb{R}^m$ are the bounded state and control spaces. The state space X contains obstacles $X_{obs} \subset X$ and the free space is denoted by $X_f = X \setminus X_{obs}$. $f : X \times U \rightarrow X$ is a Lipschitz continuous function, and x_{init} is the initial state of the system. The system behavior is dictated by differential equations of the following form:

$$\dot{x} = f(x(t), u(t)) \quad (4)$$

Where, $x(t) \in X, u(t) \in U$. we denote by $x[x_{init}, u]$ the state trajectory originating at x_{init} obtained by implementing control policy u . Let $v = \{u : \mathbb{R}_{\geq 0} \rightarrow U\}$ be the set of all control policies. The system S is said to satisfy an STL specification ϕ under a control policy $u \in v$ if the state trajectory starting at x_0 satisfies ϕ , i.e., $x[x_{init}, u] \models \phi$.

4.2 Mission Specification

In this work, we focus primarily on the development of a highly efficient trajectory planner to use in the context of a temporal logic planning problem. As such, we assume the existence of a *candidate* solution, which can be obtained via solving a MILP using a low-fidelity motion model [2, 27], applying SMT-based reasoning over an abstract syntax tree representation [3, 16], or by using a logic such as Time Window Temporal Logic [30]. Typical approaches to solving plan synthesis with a MILP use low-fidelity motion models to identify a discrete motion plan, which is then implemented by

a continuous motion planner [17]. For non-trivial motion models, a low-fidelity model may be highly inaccurate or produce infeasible trajectories. Thus, we consider an intermediate specification consisting of disjoint regions to be visited, each with its own time intervals. That is, we assume a high-level planner produces a specification Ψ such that for a given signal s , $s \models \Psi \implies s \models \phi$. However, since the high-level planner does not account for kinodynamic motion plans, satisfaction of the (simpler) specification Ψ may not be possible. Our approach is agnostic to how Ψ is obtained. Thus, Ψ is directly part of our problem formulation. Our goal is to find kinodynamic plans over a fragment of STL of the form:

$$\Psi = \left(\bigwedge_{i=1}^n \psi_b^i \right) \wedge \left(\bigwedge_{j=1}^m \psi_{un}^j \right) \quad (5)$$

$$\psi_b^i = \mathbf{F}_{[a_i, b_i]}(x \in X_{goal_i}), \quad \psi_{un}^j = \mathbf{F}_{[0, \infty)}(x \in X_{goal_j})$$

Where, ψ_b, ψ_{un} represent bounded, and unbounded goals respectively. Unbounded goals are defined over the interval $[0, \infty)$.

Problem 3.1: Given a dynamical system S and an STL specification Ψ as written in (5), find a control policy u such that the system satisfies Ψ under policy u , and the cost function (based on the robustness metric) for the state trajectory is minimized. It will be necessary to assume that the problem can be solved using trajectories generated by piecewise constant control functions. This is a reasonable way to generate a trajectory using a computational approach [18].

5 Proposed Approach

In this section, we describe how the proposed algorithm finds a kinodynamically feasible path that satisfies the specification Ψ , as outlined in Algorithm 1. First, **CandidatePlans**(Ψ) enumerates all possible orders in which the spatial regions can be visited while adhering to time constraints. Next, **GeoPlanner** generates a geometric path that visits these regions in the specified order. Finally, this geometric path is passed to our **LG-SST-STL** planner (modified **SST** planner [18]) which utilizes the geometric path as a guide for sampling, optimizing for the robustness value of the formula Ψ . If **LG-SST-STL** returns a positive robustness value, this indicates that the found kinodynamic path is valid, and the process concludes.

Algorithm 1: High Level Planner

Input: $S = (f, X, U, x_{init}), N_{max}, T_{max}, J(x, u), \Psi$

Output: Optimal Control Policy u^*

```

1  $\{M_1, M_2, \dots, M_m\} \leftarrow \text{CandidatePlans}(\Psi)$ 
2 for  $i \leftarrow 1$  to  $m$  do
3    $\{GP, L_{max}\} \leftarrow \text{GeoPlanner}(M_i)$ 
4    $\{u, \rho\} \leftarrow \text{LG-SST-STL}(\{S, GP, L_{max}\})$ 
5   if  $\rho \geq 0$  then
6     return  $u$ 
```

5.1 Discrete Region Orders

Algorithm 2 comes up with all possible orders of regions that need to be visited sequentially to satisfy the formula Ψ . In **algorithm 2, lines 2-5**, all bounded goals, denoted by ψ_b , are evaluated for potential time overlaps. If there is no overlap, one region must be

Algorithm 2: CandidatePlans

Input: Ψ

Output: $M_1, \dots, M_m$

```

1  $constraints \leftarrow \{\}$ 
2 for  $i = 1$  to  $n$  do
3   for  $j = 1$  to  $n$  do
4     if  $\text{no\_time\_overlap}(\psi_b^i, \psi_b^j)$  then
5        $constraints \leftarrow \{constraints, (i, j)\}$ 
6  $\mathcal{M} \leftarrow \{\}$ 
7 for  $M \in \text{perm}(\psi_b, \psi_{un})$  do
8   if  $\text{no\_violation}(constraints, M)$  then
9      $\mathcal{M} \leftarrow \{\mathcal{M}, M\}$ 
10 return  $\mathcal{M}$ 
```

visited strictly before the other, and these orderings are added to the set of constraints. Subsequently, in **Algorithm 2, lines 7-9**, we generate all possible permutations of the regions to be visited and check for any violations of the ordering constraints derived from the time intervals. If no constraints are violated, the permutation is added to the set of candidate paths.

This algorithm has a worst-case time complexity of $O(n!)$ in terms of the number of potential candidate paths to be planned by the motion planner. Nonetheless, as it enumerates all possible orders, it is complete. Additionally, for complex nonlinear dynamic models where computing steering functions are intractable, we argue that alternative methods encoding the full formula ϕ as a single cost function in an AO planner fail to achieve convergence to positive robustness, as demonstrated by our experimental results.

5.2 Continuous Spatial Path

The candidate sequence regions $M = \{X_{goal_1}, X_{goal_2}, \dots, X_{goal_n}\}$, to be visited, serves as a series of intermediate goals, where $X_{goal_1} = x_{init}$ is the initial state and $X_{goal_n} = x_{goal}$ is the final goal. To construct the lead geometric path GP , we utilize a geometric planner, such as Rapidly-exploring Random Trees (RRT*). The path is generated as the sum of sub-paths, where each sub-path connects two consecutive intermediate goals, $\text{GeoPlan}(X_{goal_i}, X_{goal_{i+1}})$ for $i = 1, \dots, n-1$, as described in **Algorithm 3, Line 3**. We propose

Algorithm 3: GeoPlanner

Input: M

Output: GP, L_{max}

```

1  $\{X_{goal_1}, X_{goal_2}, \dots, X_{goal_n}\} = M$ 
2  $X_{goal_1} = x_{init}, X_{goal_n} = x_{goal}$ 
3  $GP \leftarrow \sum_{i=1}^{n-1} \text{RRT}^*(X_{goal_i}, X_{goal_{i+1}})$ 
4  $L_{max} \leftarrow 2n - 1$ 
```

decomposing the global geometric path GP into sub-regions called **Layers**. We construct one layer for each subpath ($X_{goal_i}, X_{goal_{i+1}}$) and for each subgoal X_{goal_i} for a total $2n - 1$ layers. An illustrative example is shown in Figure 2, where intermediate goals and their connecting subpaths are structured into layers. These layers define regions that guide biased sampling, enforce the correct visitation order of the tree, and enable crossovers when necessary based on the sequence of regions.

5.3 Kinodynamic Spatial Temporal Path

In this section, we explain in detail the modifications we propose and outline how our framework integrates with the SST [18] planner. Modifications are highlighted in **green** in **Algorithm 4**, and the following subsections explain each modification in detail.

Algorithm 4: LG-SST-STL

Input: $S = (f, X, U, x_{init}), N_{max}, T_{max}, J(x, u), GP, L_{max}$
Output: Optimal Control Policy u^*

```

1  $\mathcal{T} = \{x_{init}\}; \text{Cost}(x_{init}) = 0$ 
2 for  $i = 1$  to  $N_{max}$  do
3    $L_{rand}, x_{rand} \leftarrow \text{SpatialBiasSampler}(X, L_{max})$ 
4    $x_{near} \leftarrow \text{NearestNeighbor}(\mathcal{T}, x_{rand})$ 
5    $u_{rand} \leftarrow \text{Sample}(U)$ 
6    $T_{rand} \leftarrow \text{Sample}(0, T_{max})$ 
7    $x[t] \leftarrow x_{near} + \int_0^{T_{rand}} f(x(\tau), u_{rand}) d\tau$ 
8   if  $x[t] \in X_f$  then
9     if  $\text{dist}(x[t] - GP) \leq r_{prop}$  then
10       $L(x[t]) \leftarrow \text{LayerAssign}(x[t])$ 
11      if  $|L(x_{near}) - L(x[t])| \leq 1$  then
12         $\text{newCost} \leftarrow \text{Cost}(x_{near}) + J(x[t], u_{rand})$ 
13        if  $\text{newCost} < \text{Cost}(x[t])$  or  $x[t] \notin \mathcal{T}$  then
14           $\text{AddNode}(\mathcal{T}, x[t])$ 
15           $\text{Cost}(x[t]) \leftarrow \text{newCost}$ 
16           $\text{UpdateTree}(\mathcal{T}, x[t])$ 
17   if  $\text{IsGoalReached}(x[t], x_{goal})$  then
18     return  $\text{ReconstructPath}(\mathcal{T}, x[t]), u[t]$ 
19 return Failure

```

5.3.1 Biased Sampling. We use the global geometric path GP to bias the sampling strategy. This strategy confines sampling to regions proximal to the global path. To control this proximity, we introduce a new parameter called the **sampler selection radius** (s_r), which restricts sampling to a neighborhood around GP . The sampling process is described in **Algorithm 5**, specifically **Line 2**, and is illustrated in Figure 2 which is the outer layer shaded in the color orange. By focusing sampling efforts along the guidance path, the planner achieves higher computational efficiency. This is because the geometric path provides a partial solution for the spatial component of the STL specification, significantly reducing the search space.

Algorithm 5: Spatial Bias Sampler

Input: X, L_{max}
Output: x_{rand}, L_{rand}

```

1  $L_{rand} \leftarrow \text{Sample}(0, L_{max})$ 
2  $x_{rand} \leftarrow \text{BiasSampler}(X, s_r)$ 

```

5.3.2 Layer Assignment. The layers assigned to the global path earlier are now used to guide the planner during sampling and tree growth. Specifically, each sampling iteration selects a layer, L_{rand} , and a sample, x_{rand} , within that layer as shown in **Algorithm 5**, **Line 1**. This ensures that the planner grows the tree within the

selected layer, as illustrated in Figure 2. Layers are also assigned to each node in the planner's tree based on proximity to the corresponding layer of GP . This assignment is described in **Algorithm 6**, **Line 1**. A new node, $x[t]$, is only added to the tree if it belongs to a layer consecutive to its parent node, as enforced in **Algorithm 4**, **Line 11**. This restriction ensures that the planner maintains consistency with the STL specification and avoids undesired behavior by connecting with incorrect nodes in the tree.

Algorithm 6: Layer Assignment

Input: $x[t]$
Output: $L(x[t])$

```

1  $L(x[t]) \leftarrow \arg \min_{l \in \{1, \dots, L_{max}\}} \text{dist}(x[t], GP_l)$ 

```

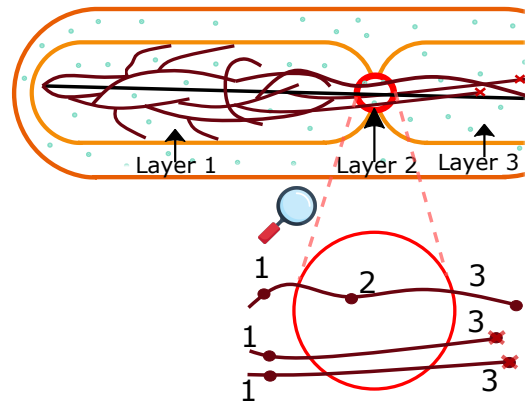


Figure 2: Layer Assignment and connection of nodes, the magnified image shows the node that are rejected.

5.3.3 Propagation Radius. We introduce a new parameter, the **propagation allowed radius** (r_{prop}), which limits the planner's growth to remain within a defined radius r the global path GP . This restriction is enforced in **Algorithm 4**, **Line 9**. If a propagated node $x[t]$ exceeds r_{prop} , it is rejected, as shown in Figure 3. By limiting deviations from the global path, the propagation radius ensures that the planner remains focused on regions critical for satisfying STL constraints. Adjusting r_{prop} can help balance between exploration and adherence to the guidance path.

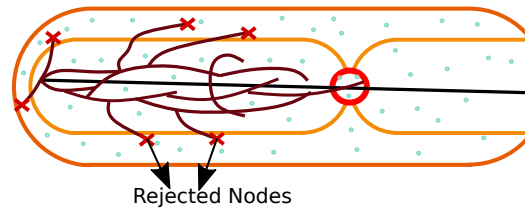


Figure 3: The figure shows rejection of nodes that deviate more than allowed by the propagation radius

5.3.4 Satisfy the Temporal constraints (Cost Function). The above methodology will help us create a kinodynamic path that will satisfy all the spatial components of the STL formulae by design, and it will additionally be time-parameterized. Cost functions have been used before in many different scenarios, such as [11, 19, 31]. We will be using a simple cost inspired by [19] that will ensure that as it decreases, the STL specification will be satisfied. We define a simple cost function J that includes the robustness of an STL specification. For calculating cost after each new node that is added to the tree $\mathcal{T}$, we make the cost compute recursive, which helps reduce overhead.

$$\bar{\rho}(\mathbf{x}_i, (f(\xi_i) \sim \mu), t) = \begin{cases} \mu - f(\xi_i(t)) & \sim = \leq \\ f(\xi_i(t)) - \mu & \sim = \geq \end{cases} \quad (6)$$

$$\bar{\rho}(\mathbf{x}_i, \phi_1 \wedge \phi_2, t) = \begin{cases} \star & \text{if } \bar{\rho}(\mathbf{x}_i, \phi_1, t) \text{ and } \bar{\rho}(\mathbf{x}_i, \phi_2, t) = \star \\ \bar{\rho}(\mathbf{x}_i, \phi_1, t) & \text{if } \bar{\rho}(\mathbf{x}_i, \phi_2, t) = \star \\ \bar{\rho}(\mathbf{x}_i, \phi_2, t) & \text{if } \bar{\rho}(\mathbf{x}_i, \phi_1, t) = \star \\ \min(\bar{\rho}(\mathbf{x}_i, \phi_1, t), \bar{\rho}(\mathbf{x}_i, \phi_2, t)) & \end{cases} \quad (7)$$

$$\bar{\rho}(\mathbf{x}_i, F_{[a,b]}\phi, t) = \begin{cases} \star & \text{if } t < a \text{ or } t > b \\ \bar{\rho}(\mathbf{x}_i, \phi, t) & \text{if } t = a \\ \max(\bar{\rho}(\mathbf{x}_i, \phi, t), \bar{\rho}_{\text{parent}}(\mathbf{x}_i, F_{[a,b]}\phi)) & \end{cases} \quad (8)$$

$$\bar{\rho}(\mathbf{x}_i, F\phi, t) = \max(\bar{\rho}(\mathbf{x}_i, \phi, t), \bar{\rho}_{\text{parent}}(\mathbf{x}_i, F\phi)) \quad (9)$$

Hereinafter, we define the robustness value $\bar{\rho}$ associated to a node $\mathbf{x}_i$ in $\mathcal{T}$. Since a trajectory until a given node in the LG-SST-STL tree might be partially testable against the STL specification, we recursively define the robustness of (partial) trajectories until node $\mathbf{x}_i$. Consider a simple specification like $F_{[4,5]}(x > 2)$. It is not possible to assess the trajectory's robustness against the specification until the first observation of the trajectory for time $t = 4$ has been made. As highlighted in [4], we use and maintain a syntax tree of STL formula in memory, augmented with a robustness value $\bar{\rho}$ associated with the nodes in $\mathcal{T}$. Temporal operators are equipped with robustness values $\bar{\rho}$ for nodes $\mathbf{x}_i \in \mathcal{X}$, that are stored. Further, we define by $\bar{\rho}_{\text{parent}} : \mathcal{X} \times \Phi^{\Sigma} \rightarrow \mathbb{R} \cup \{\star\}$ the $\bar{\rho}$ value of a temporal operator in the syntax tree of the specification, for the parent of node $\mathbf{x}_i$ in the LG-SST-STL tree, where $\star$ is a dummy symbol used to provide no real value to a formula, the robustness of which cannot be stated for a given trajectory (i.e., for trajectories shorter than the lower bound of the interval of a temporal operator). In the following, $\bar{\rho}_{\text{parent}}$ is called to compute the actual value of $\bar{\rho}$ for node $\mathbf{x}_i$ (since $\bar{\rho}$ depends on the value of $\bar{\rho}$ for the parent node of $\mathbf{x}_i$). The robustness value $\bar{\rho} : \mathcal{X} \times \Phi^{\Sigma} \times \mathbb{N} \rightarrow \mathbb{R} \cup \{\star\}$ associated to a node $\mathbf{x}_i$ in $\mathcal{T}$ is given by the recursive function:

Timed temporal operators, the design of $\bar{\rho}$ only uses the evaluation of predicates for the relevant time intervals. Outside of these, predicates are not evaluated, hence are not reflected in the calculation of the cost function. Also, the definition of $\bar{\rho}$ as such enables an easy computation of the min and max in the case of temporal operators: for a given node $\mathbf{x}_i$, the whole trajectory until node $\mathbf{x}_i$ doesn't need to be tested, but only the spatial coordinates of node

$\mathbf{x}_i$ and the value of $\bar{\rho}$ of the temporal operator for the parent of $\mathbf{x}_i$, which leads to lower computational complexity. We now define $\bar{\rho} : \mathcal{X} \times \Phi^{\Sigma} \times \mathbb{N} \rightarrow \mathbb{R}$, that will be directly called by the LG-SST-STL cost function: $J(\mathbf{x}_i) = J_{\phi}(\mathbf{x}_i) = \bar{\rho}(\mathbf{x}_i, \phi, t) = -\min(\bar{\rho}(\mathbf{x}_i, \phi, t), 0)$

6 Case Studies

We validate our approach through simulations conducted using the Open Motion Planning Library (OMPL)[29], for our implementation [13]. Our experiments encompass three environments, each designed to test an Ackermann-steered vehicle navigating in $SE(2)$ configuration space, where the state vector $\mathbf{x} = [x, y, \theta]^T \in \mathbb{R}^2 \times \mathbb{S}^1$ represents the vehicle's position and orientation. The system dynamics follow the standard Ackermann-steering model: $\dot{x} = v \cos(\theta)$, $\dot{y} = v \sin(\theta)$, $\dot{\theta} = \frac{v}{L} \tan(\delta)$ where v is the linear velocity, L is the wheelbase length, and $\delta \in [-\delta_{\max}, \delta_{\max}]$ is the steering angle. Each environment presents unique challenges and incorporates different STL specifications. For the experiments, the intermediary goal threshold ε is set to 0.3 m, ensuring the vehicle comes sufficiently close to each goal region. We evaluate our approach against the baseline SST planner with STL cost using OMPL's benchmarking tools [21]. The experimental evaluation consists of 60 runs for each planner across three distinct environments. For each run, we analyze the evolution of the STL cost J over a time horizon of 300 seconds, along with the computational resources required by each planner. The comparative results of both performance metrics are presented in Figure 4.

Experiment 1: The Objective of Experiment 1 is to evaluate the capability of the SST-STL planner and the proposed LG-SST-STL planner in sequentially achieving two unbounded spatial goals without incorporating time bounds into the cost function. This experiment assesses the planners' ability to generate feasible paths that satisfy the given STL specifications.

$$\Psi = F(d((x, y) - (5, 4)) \leq \varepsilon) \wedge F(d((x, y) - (10, 4)) \leq \varepsilon)$$

As depicted in Figure 4, both the SST-STL planner and the LG-SST-STL planner successfully converged to feasible solutions, generating paths that satisfy the specified STL fragment.

Experiment 2: consists of four sequential, time-bounded goals designed to evaluate the planner's ability to handle temporal constraints in navigation in presence of obstacle

$$\begin{aligned} \Psi = & F_{[0,3]}(d((x, y) - (0.5, 4)) \leq \varepsilon) \wedge \\ & F_{[6,20]}(d((x, y) - (5, 4)) \leq \varepsilon) \wedge \\ & F_{[20,40]}(d((x, y) - (10, 4)) \leq \varepsilon) \wedge \\ & F_{[35,65]}(d((x, y) - (10, 1)) \leq \varepsilon), \end{aligned}$$

As shown in 4B, the SST planner with STL cost fails to find a path that meets all requirements. The planner opts for a longer path around the obstacle. This inefficiency prevents the planner from reaching some goals within respective time bounds, thus only partial satisfying STL. In contrast, the LG-SST-STL planner demonstrates significant improvements. By selecting the shortest route to Goal 1, it satisfies the time constraint and sequentially meets the deadlines for Goals 2 and 3. Notably, this approach requires ten times fewer graph states than the SST baseline.

Experiment 3: In this experiment, we demonstrate the planner's ability to handle crossovers. This capability is achieved through

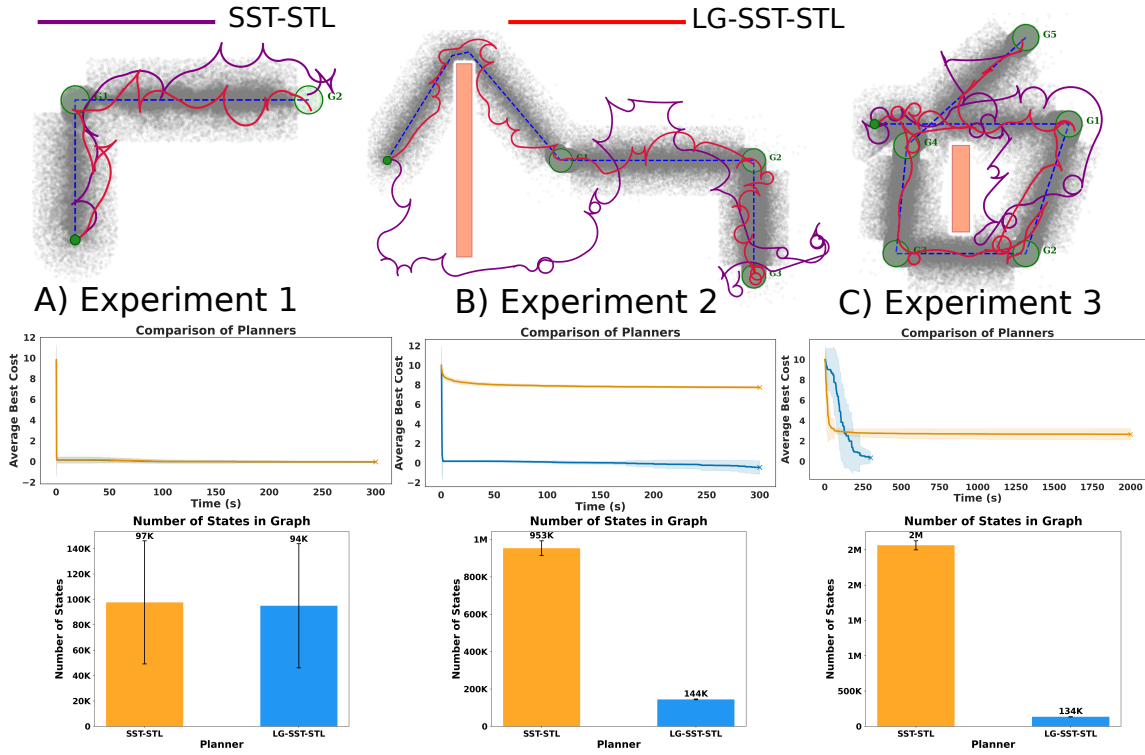


Figure 4: Comparison of the SST-STL and LG-SST-STL planners across three experiments. The middle row displays the average best cost achieved by each planner over time. The bottom row compares the number of states in the graph.

our layer based strategy, which restricts node connections to adjacent layers, effectively addressing the challenges posed by loops in path planning. The environment includes multiple goals with overlapping time bounds, creating a scenario where the order of visiting certain goals does not impact the overall satisfaction of the temporal logic specification. Due to the overlapping time bounds of multiple goals, the order in which these goals are visited is interchangeable, provided that each is reached within its time bounds. This results in multiple feasible sequences, and the planner must consider all permutations to find an optimal path.

$$\begin{aligned}
 \Psi = & \mathbf{F}_{[0,2]}(d((x, y) - (0.5, 4)) \leq \epsilon) \wedge \\
 & \mathbf{F}_{[6,20]}(d((x, y) - (5, 4)) \leq \epsilon) \wedge \\
 & \mathbf{F}_{[20,30]}(d((x, y) - (4, 1)) \leq \epsilon) \wedge \\
 & \mathbf{F}_{[25,90]}(d((x, y) - (1, 1)) \leq \epsilon) \wedge \\
 & \mathbf{F}_{[30,120]}(d((x, y) - (1.25, 3.5)) \leq \epsilon) \wedge \\
 & \mathbf{F}_{[35,150]}(d((x, y) - (4, 6)) \leq \epsilon),
 \end{aligned}$$

Even though our method needs to be run for each of $3! = 6$ permutations, it still requires less time and converges faster than the baseline planner. Our layer-guided SST planner efficiently navigates through these permutations by leveraging the geometric path, which reduces the search space and computational complexity.

7 Conclusion

We developed a new approach to motion planning that efficiently handles spatiotemporal constraints. Our experiments demonstrated that our method significantly reduces computation time while handling complex scenarios with time-bounded goals. Notably the proposed method, can even produce crossover paths, if required by the specification. Future work will explore more expressive STL constraints, dynamic obstacles, and multi-robot systems.

Acknowledgments

The authors thank **Michael Da Silva** for his help in creating the diagrams that appear in this manuscript.

References

- [1] Fernando S Barbosa, Daniel Duberg, Patric Jensfelt, and Jana Tumova. 2019. Guiding Autonomous Exploration With Signal Temporal Logic. *IEEE Robotics and Automation Letters* 4, 4 (oct 2019), 3332–3339. <https://doi.org/10.1109/lra.2019.2926669>
- [2] Calin Belta and Sadra Sadraddini. 2019. Formal methods for control synthesis: An optimization perspective. *Annual Review of Control, Robotics, and Autonomous Systems* 2, 1 (2019), 115–140.
- [3] Gustavo A Cardona, Kevin Leahy, Makai Mann, and Cristian-Ioan Vasile. 2023. A flexible and efficient temporal logic tool for python: PyTeLo. *arXiv preprint arXiv:2310.08714* (2023).
- [4] Jyotirmoy V. Deshmukh, Alexandre Donzé, Shromona Ghosh, Xiaoping Jin, Garvit Juniwal, and Sanjit A. Seshia. 2015. Robust Online Monitoring of Signal Temporal Logic. (2015). [arXiv:1506.08234 \[cs.SY\]](https://arxiv.org/abs/1506.08234) <https://arxiv.org/abs/1506.08234>
- [5] Alexandre Donzé and Oded Maler. 2010. Robust satisfaction of temporal logic over real-valued signals. In *International Conference on Formal Modeling and*

- Analysis of Timed Systems*. Springer, 92–106.
- [6] Georgios E Fainekos and George J Pappas. 2009. Robustness of temporal logic specifications for continuous-time signals. *Theoretical Computer Science* 410, 42 (2009), 4262–4291.
- [7] Jonathan D Gammell and Marlin P Strub. 2021. Asymptotically optimal sampling-based motion planning methods. *Annual Review of Control, Robotics, and Autonomous Systems* 4, 1 (2021), 295–318.
- [8] Caelan Reed Garrett, Rohan Chitnis, Rachel Holladay, Beomjoon Kim, Tom Silver, Leslie Pack Kaelbling, and Tomás Lozano-Pérez. 2021. Integrated task and motion planning. *Annual review of control, robotics, and autonomous systems* 4, 1 (2021), 265–293.
- [9] Qi Heng Ho, Roland B. Ilyes, Zachary N. Sunberg, and Morteza Lahijanian. 2022. Automaton-Guided Control Synthesis for Signal Temporal Logic Specifications. In *2022 IEEE 61st Conference on Decision and Control (CDC)*. 3243–3249. <https://doi.org/10.1109/CDC51059.2022.9993090>
- [10] Sertac Karaman and Emilio Frazzoli. 2011. Sampling-based Algorithms for Optimal Motion Planning. *International Journal of Robotics Research* 30, 7 (2011), 846–894.
- [11] Jesper Karlsson, Fernando S. Barbosa, and Jana Tumova. 2020. Sampling-based Motion Planning with Temporal Logic Missions and Spatial Preferences. *IFAC-PapersOnLine* 53, 2 (2020), 15537–15543. <https://doi.org/10.1016/j.ifacol.2020.12.2397> 21st IFAC World Congress.
- [12] Hadas Kress-Gazit, Morteza Lahijanian, and Vasumathi Raman. 2018. Synthesis for robots: Guarantees and feedback for robot behavior. *Annual Review of Control, Robotics, and Autonomous Systems* 1, 1 (2018), 211–236.
- [13] ELPIS Lab. 2025. *LG-SST-STL: Layer-Guided Sparse Stable Trees for Signal Temporal Logic*. <https://github.com/elpis-lab/LG-SST-STL>
- [14] Steven M LaValle. 2006. *Planning algorithms*. Cambridge university press.
- [15] Steven M. LaValle and James J. Kuffner. 2001. Randomized Kinodynamic Planning. *International Journal of Robotics Research* 20, 5 (2001), 378–400.
- [16] Kevin Leahy, Austin Jones, and Cristian-Ioan Vasile. 2022. Fast decomposition of temporal logic specifications for heterogeneous teams. *IEEE Robotics and Automation Letters* 7, 2 (2022), 2297–2304.
- [17] Kevin Leahy, Zachary Serlin, Cristian-Ioan Vasile, Andrew Schoer, Austin M Jones, Roberto Tron, and Calin Belta. 2021. Scalable and robust algorithms for task-based coordination from high-level specifications (scratches). *IEEE Transactions on Robotics* 38, 4 (2021), 2516–2535.
- [18] Yanbo Li, Zakary Littlefield, and Kostas E. Bekris. 2016. Asymptotically optimal sampling-based kinodynamic planning. *The International Journal of Robotics Research* 35, 5 (2016), 528–564. <https://doi.org/10.1177/0278364915614386> arXiv:<https://doi.org/10.1177/0278364915614386>
- [19] Alexis Linard, Ilaria Torre, Ermanno Bartoli, Alex Sleat, Iolanda Leite, and Jana Tumova. 2023. 2023 IEEE/RSJ international conference on intelligent robots and systems (IROS) :. <https://iee-iro.org/QC20231122>.
- [20] Oded Maler and Dejan Nickovic. 2004. Monitoring Temporal Properties of Continuous Signals. (2004).
- [21] Mark Moll, Ioan A. Şucan, and Lydia E. Kavraki. 2015. Benchmarking Motion Planning Algorithms: An Extensible Infrastructure for Analysis and Visualization. *IEEE Robotics & Automation Magazine* 22, 3 (September 2015), 96–102. <https://doi.org/10.1109/MRA.2015.2448276>
- [22] Andreas Orthey, Constantinos Chamzas, and Lydia E. Kavraki. 2024. Sampling-Based Motion Planning: A Comparative Review. *Annual Review of Control, Robotics, and Autonomous Systems* 7, 1 (July 2024), 285–310. <https://doi.org/10.1146/annurev-control-061623-094742>
- [23] Joaquim Ortiz-Haro, Wolfgang Hönig, Valentin N Hartmann, Marc Toussaint, and Ludovic Righetti. 2024. iDb-RRT: Sampling-based Kinodynamic Motion Planning with Motion Primitives and Trajectory Optimization. *arXiv preprint arXiv:2403.10745* (2024).
- [24] Erion Plaku. 2012. Planning in discrete and continuous spaces: From LTL tasks to robot motions. In *Advances in Autonomous Robotics: Joint Proceedings of the 13th Annual TAROS Conference and the 15th Annual FIRA RoboWorld Congress, Bristol, UK, August 20-23, 2012*. Springer, 331–342.
- [25] Erion Plaku, Lydia E. Kavraki, and Moshe Y. Vardi. 2010. Motion Planning With Dynamics by a Synergistic Combination of Layers of Planning. *IEEE Transactions on Robotics* 26, 3 (2010), 469–482. <https://doi.org/10.1109/TRO.2010.2047820>
- [26] Amir Pnueli. 1977. The Temporal Logic of Programs. In *18th Annual Symposium on Foundations of Computer Science*. 46–57.
- [27] Vasumathi Raman, Mehdi Maasoumy, and Alexandre Donzé. 2014. Model predictive control from signal temporal logic specifications: A case study. In *Proceedings of the 4th ACM SIGBED International Workshop on Design, Modeling, and Evaluation of Cyber-Physical Systems*. 52–55.
- [28] Rahul Shome and Lydia E. Kavraki. 2021. Asymptotically Optimal Kinodynamic Planning Using Bundles of Edges. In *IEEE International Conference on Robotics and Automation (ICRA)*. 9988–9994. <https://doi.org/10.1109/ICRA48506.2021.9560836>
- [29] Ioan A. Şucan, Mark Moll, and Lydia E. Kavraki. 2012. The Open Motion Planning Library. *IEEE Robotics & Automation Magazine* 19, 4 (December 2012), 72–82. <https://doi.org/10.1109/MRA.2012.2205651> <https://ompl.kavrakilab.org>.
- [30] Cristian-Ioan Vasile, Derya Aksaray, and Calin Belta. submitted. Time Window Temporal Logic. *Theoretical Computer Science* (submitted). (linkunavailable)
- [31] Cristian-Ioan Vasile, Vasumathi Raman, and Sertac Karaman. 2017. Sampling-based synthesis of maximally-satisfying controllers for temporal logic specifications. In *2017 IEEE/RSJ International Conference on Intelligent Robots and Systems (IROS)*. 3840–3847. <https://doi.org/10.1109/IROS.2017.8206235>
- [32] Ioan A. Şucan and Lydia E. Kavraki. 2010. Kinodynamic Motion Planning by Interior-Exterior Cell Exploration. In *Algorithmic Foundation of Robotics VIII: Selected Contributions of the Eight International Workshop on the Algorithmic Foundations of Robotics*, Gregory S. Chirikjian, Howie Choset, Marco Morales, and Todd Murphey (Eds.). Springer Berlin Heidelberg, Berlin, Heidelberg, 449–464. https://doi.org/10.1007/978-3-642-00312-7_28

Received 14 November 2024; revised 10 March 2025